CaçaPreço Games

Comparador de preços de jogos com histórico, alerta de promoção e indicador de 'vale esperar?'

1. O problema (real e pequeno)

O mesmo jogo tem preços diferentes em cada loja digital, os descontos aparecem e somem rápido e é difícil saber se uma promoção é boa de verdade ou se o jogo costuma ficar mais barato depois. Quem compra jogos acaba pesquisando loja por loja.

Quem sofre com isso: jogadores de PC e console. Para o mercado: pode gerar receita com links de afiliado das lojas e parcerias com comunidades de games.

2. A solução proposta

O usuário digita o nome do jogo e vê a lista de lojas (Steam, Epic Games, GOG, Humble, Fanatical e outras) **ordenada do preço mais baixo ao mais alto, em reais**. Além do comparador, a plataforma guarda o histórico de preços, avisa quando o jogo atingir o preço desejado e indica se vale comprar agora ou esperar.

Principais funcionalidades:

- Busca por nome com lista de lojas ordenada pelo menor preço, já convertido para reais.
- Histórico de preços por loja, com menor preço já registrado e média.
- Indicador 'vale esperar?': compara o preço atual com o histórico e as grandes promoções sazonais.
- Wishlist e alerta de preço desejado (notificação quando o preço cair).
- Preço por hora de jogo: relaciona o preço com a duração média do jogo.
- Comparação do custo total da wishlist: qual loja fica mais barata para o carrinho inteiro.

3. Fontes de dados e integração

Fonte	Tipo / formato	O que traz
CheapShark API	REST / JSON	Ofertas e preços de várias lojas de jogos para PC.
Steam Store API	REST / JSON	Preço em reais direto da Steam (região Brasil) e detalhes do jogo.
RAWG API	REST / JSON	Dados do jogo: capa, gênero, nota, plataformas e tempo médio de jogo (chave gratuita).
API do Banco Central (PTAX)	REST / JSON	Cotação do dólar para converter preços em USD para reais.
Calendário de promoções	Tabela interna	Datas das grandes liquidações sazonais das lojas.

4. Arquitetura

Fluxo geral: Fontes > Integração (ETL) > Armazenamento > API Python > Interface React

Fontes	Integração	Armazenamento	Backend	Frontend
CheapShark, Steam, RAWG, Banco Central	KNIME (workflows agendados)	PostgreSQL + MongoDB + Firebase	Python + FastAPI	React + Recharts

Fluxo de integração (pipeline):

- Workflow 1 (KNIME): a cada poucas horas consulta CheapShark e Steam para os jogos acompanhados e grava o preço do dia por loja no PostgreSQL.

- Workflow 2 (KNIME): consulta o PTAX no Banco Central e atualiza a cotação do dólar usada na conversão.
- Workflow 3 (KNIME): busca os dados dos jogos na RAWG e guarda o documento de cada jogo no MongoDB.
- API Python: responde as buscas, calcula o indicador 'vale esperar?', dispara os alertas e usa o Firebase para contas, wishlist e notificações.

5. Tecnologias utilizadas

Tecnologia	Papel no projeto	Por que foi escolhida
PostgreSQL	Histórico de preços por jogo, loja e data; cotação do dólar.	Séries temporais e consultas por período (mínimo, média e tendência).
MongoDB	Catálogo de jogos com metadados variados vindos da RAWG e das lojas.	Cada jogo traz campos diferentes (plataformas, gêneros, edições).
Firebase (Auth + Firestore + Cloud Messaging)	Login dos usuários, wishlist, alertas de preço e notificações push.	Autenticação pronta e atualização em tempo real para o usuário.
KNIME	ETL agendado das APIs de preços, câmbio e catálogo.	Fluxos visuais, fáceis de demonstrar.
Python + FastAPI	Busca, conversão para reais, ordenação, regra do 'vale esperar?' e alertas.	Backend simples e rápido.
React + Recharts	Tela de busca, lista ordenada por preço e gráfico de histórico.	Interface intuitiva e responsiva.
Docker Compose	Ambiente reproduzível.	Facilita a demonstração.

6. Telas do frontend (React)

- Home com campo de busca grande e jogos em promoção.
- Resultado: lista de lojas do mais barato ao mais caro, com link para a loja.
- Página do jogo: gráfico de histórico, menor preço, indicador 'vale esperar?' e tempo médio de jogo.
- Wishlist com alertas e o custo total por loja.

7. Onde está a criatividade

- Vai além do comparador: responde se é o melhor momento de comprar.
- Preço por hora de jogo ajuda a comparar jogos de tamanhos diferentes.
- Alerta personalizado por preço-alvo, com notificação no celular.

8. Checklist dos requisitos do desafio

Requisito	Como é atendido
3+ tecnologias de armazenamento	PostgreSQL, MongoDB e Firebase.
Ferramenta de integração	KNIME (3 workflows).
API externa	CheapShark, Steam, RAWG e Banco Central.
Backend Python / Frontend React	FastAPI e React.
Consolidação de dados	Preços de várias lojas + câmbio + catálogo + histórico em uma única visão.
Interface e demonstração	Busca, lista ordenada, histórico e alerta de preço.

9. Roteiro sugerido para a apresentação (5 min)

- 0:00-0:45 Problema: pesquisar o mesmo jogo em várias lojas.
- 0:45-1:45 Arquitetura: KNIME, três bancos e as quatro APIs.
- 1:45-4:00 Demonstração: buscar um jogo, ver a lista ordenada, o histórico e criar um alerta.
- 4:00-5:00 Modelo de afiliados e próximos passos (lojas de console).

10. Plano de execução sugerido

- Semana 1: modelagem, workflow de preços (CheapShark e Steam) e histórico no PostgreSQL.
- Semana 2: câmbio, catálogo na RAWG, API de busca e regra do 'vale esperar?'.
- Semana 3: frontend, login e alertas no Firebase, documentação e ensaio.
- Observação: nem todas as lojas têm API pública (por exemplo, as de console); o escopo começa pelas lojas de PC e as de console ficam como evolução.